PORTFOLIO PROJECT 06 · .NET & SQL SERVER

incident-sla-tracker

.NET 10
SQL Server 2022

An ASP.NET Core API that tracks incidents against their SLA and links them to the releases that caused them — reports computed in T-SQL, tested on real SQL Server.

C# · .NET 10 · ASP.NET Core · EF Core 10 · SQL Server 2022 · T-SQL · xUnit · Testcontainers · Docker · GitHub Actions

Sallehuddin Mohd Zakinuddin · Senior Application Developer · Portfolio project on GitHub

1 Motivation

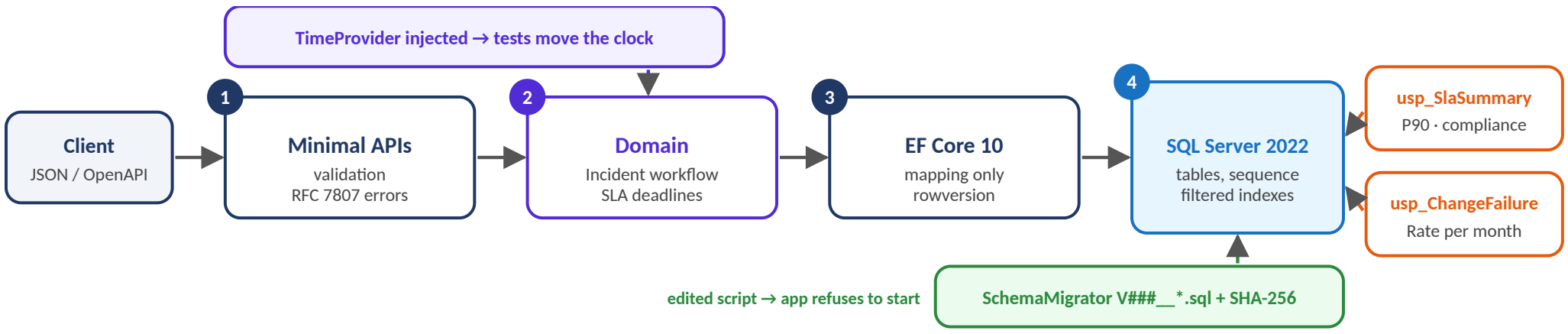
Operations teams are judged on **how fast they fix incidents** (SLA) and **how often releases break production** (change failure rate). Both usually live in spreadsheets filled in afterwards.

This project records both as they happen and computes them in SQL Server. It is also my deliberate step into the **Microsoft stack** — C#, ASP.NET Core, EF Core and T-SQL — built with the same habits as my Java work. *Data fictional.*

2 Objectives

1. Deadlines derived from severity; breaches **computed, never typed in**.
2. A workflow no caller can bypass, with a full timeline.
3. **Versioned T-SQL** applied automatically, protected against edits.
4. Reports as **stored procedures** with indexes that suit them.
5. Concurrent edits detected; tests on **real SQL Server**.

3 Architecture



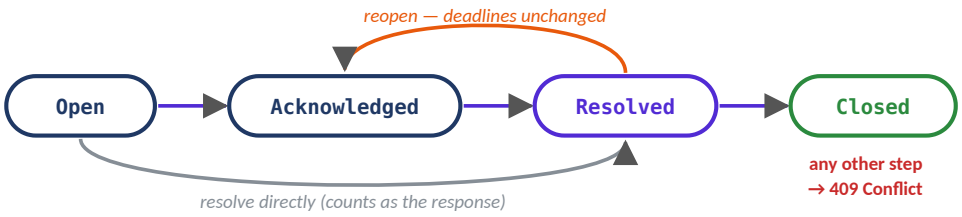
① **API.** Minimal API groups; field errors as `ValidationProblemDetails`; 404 / 409 mapped centrally.

② **Domain.** `Incident.Acknowledge/Resolve/Close/Reopen` enforce the workflow and write the timeline.

③ **EF Core.** Maps onto the scripted schema; `rowversion` turns a lost update into a 409.

④ **SQL Server.** `INC-1000…` from a `SEQUENCE`; filtered index on unresolved incidents.

4 Incident Workflow & SLA



P1: respond 15 min, resolve 4 h · P2: 30 min / 8 h
P3: 4 h / 2 days · P4: 1 day / 5 days (dbo.SlaPolicy)

Deadlines are stored at open time: a policy change never rewrites history.

5 T-SQL That Does the Work

```
-- P90 resolve time per severity, inside usp_SlaSummary
SELECT DISTINCT Severity,
  PERCENTILE_CONT(0.9) WITHIN GROUP (ORDER BY ResolveMinutes)
    OVER (PARTITION BY Severity) AS P90ResolveMinutes
FROM Scoped WHERE ResolveMinutes IS NOT NULL;

-- only unresolved rows: small index for "what breaches next"
CREATE INDEX IX_Incident_Open_ResolveDue ON dbo.Incident (ResolveDueAt)
  INCLUDE (Ref, Severity, Status)
  WHERE Status IN ('Open', 'Acknowledged');
```

Gotchas handled: no subquery inside `SUM` (CTE + join instead); `NEXT VALUE FOR` is not allowed in a derived table, so EF must not compose that query.

6 Reports in Action

```
GET /api/reports/sla?from=2031-03-01&to=2031-03-03
200 [{ "severity": "P2", "total": 2, "stillOpen": 0,
       "responseCompliancePct": 50.0, "resolveCompliancePct": 50.0,
       "avgResolveMinutes": 360.0, "p90ResolveMinutes": 552.0 }]

GET /api/reports/change-failure-rate
200 [{ "month": "2032-06", "deployments": 3, "failed": 2,
       "changeFailureRatePct": 66.7 }]

POST /api/incidents/INC-1003/close   (still Open)
409 { "title": "Not allowed",
      "detail": "Cannot close an incident that is Open; it must be Resolved." }
```

7 Testing Strategy

12	10
unit cases	SQL Server tests

- **Testcontainers** starts real SQL Server 2022.
- Clock moved hours ahead to test breaches and P90.
- Migration tamper, rowversion conflict, filtered index.

Status: syntax-checked here; first compile and test run is the GitHub Actions build.

8 Key Concepts

- The **schema is code** — reviewed T-SQL, checksummed.
- **Rules live in the domain**, not in endpoints.
- Measure what ops is judged on: **SLA, change failure rate**.

9 Limitations & Next

- 24×7 clock → business-hours calendars.
- No login → roles per step.
- React dashboard; deadline alerts.

10 Interview Talking Points

Q: Why not EF migrations?
Procedures, sequences and filtered indexes belong in reviewed T-SQL.

Q: What does rowversion do?
A stale update affects 0 rows → EF throws → 409.

Portfolio project · fictional data · MIT licence